I Ran Autonoma and KISS Sorcar on the Same Project Again. The Cheaper One Won

In Phase 1 I wrote ~300 Playwright tests by hand across a microservices SaaS. For Phase 2 I manually covered all 13 OrangeHRM modules on version 5.4. Then the upgrade to 5.8.1 broke selectors across half the suite. For Phase 3 I needed tools that could regenerate broken tests, not just write new ones — and measure cost, hallucination rate, and auto-repair.

I ran two AI testing tools on the same OrangeHRM 5.8.1 instance across three sessions, spending ~$5.63 of a $6 OpenRouter budget. **Autonoma (gpt-4o) produced 28 .md test specs for $1.18. KISS Sorcar produced 3 executable .ts tests for $0.19 — and when 2 of them failed on first run, it auto-fixed and re-ran until green. Autonoma (deepseek) cost $3.83 for 4/6 pipeline steps before context overflow and silent model-switching stopped it**. The cheaper tool won — but not for the reasons you'd expect.

1. Why Phase 3?

The first two phases of this project proved:

- Phase 1 (Buzzhive): ~300 tests, 94% API coverage, CI/CD quality gates — built manually across 26 hours
- Phase 2 (OrangeHRM 5.4): 34 tests covering all 13 modules in ~8 hours — also manual

Then the upgrade OrangeHRM from 5.4 to 5.8.1 changed selectors, routes, and UI components. Half the suite needed rewriting. That's when I realised: manual test authoring doesn't scale when your app moves.

Previously in this series: **One agent discovered 13 modules, the other wrote working code**

For Phase 3, I wanted to stress-test two AI-native testing tools on the same project and measure:

- Cost per working test
- Output format (spec vs executable)
- Hallucination rate and auto-repair capability
- Total human time invested

Setup:

- OrangeHRM 5.8.1 local Docker instance at **localhost:8080**
- OpenRouter balance: $6 (free tier unlocked at $5 spend)
- Models: Autonoma used gpt-4o for KB (~$1.18, hit budget limit), gpt-4o-mini for test generation (~$0.02); KISS used deepseek-v3.2
- Target module: Maintenance (password authentication gate, purge records search)

2. Autonoma: $1.18 for 28 Specs

The Autonoma pipeline runs in 6 steps: pagesFinder → kb (Knowledge Base) → entityAudit → scenarioRecipe → recipeBuilder → testGenerator → review.

What went well

- **Environment Factory**: Autonoma's data seeding protocol (discover/up/down) worked perfectly with OrangeHRM's PHP API. Six entity types (employee, system user, candidate, leave request, buzz post, buzz like) created and destroyed cleanly.
- **Pipeline speed**: Full run from pagesFinder to testGenerator took ~3 minutes of AI time (excluding human feedback).

Review pass: All 30/30 generated test specs passed Autonoma's built-in review (subjective pass/fail).

What went wrong

KB hallucinated 4 fake modules: Analytics — Overview, Analytics — Revenue, Analytics — Users, Analytics — Retention — none exist in OrangeHRM.

Routes regressed on feedback: When I asked the KB to fix Analytics, it regenerated the *entire* file from scratch and replaced /web/index.php/... routes with generic /dashboard, /recruitment, /buzz.

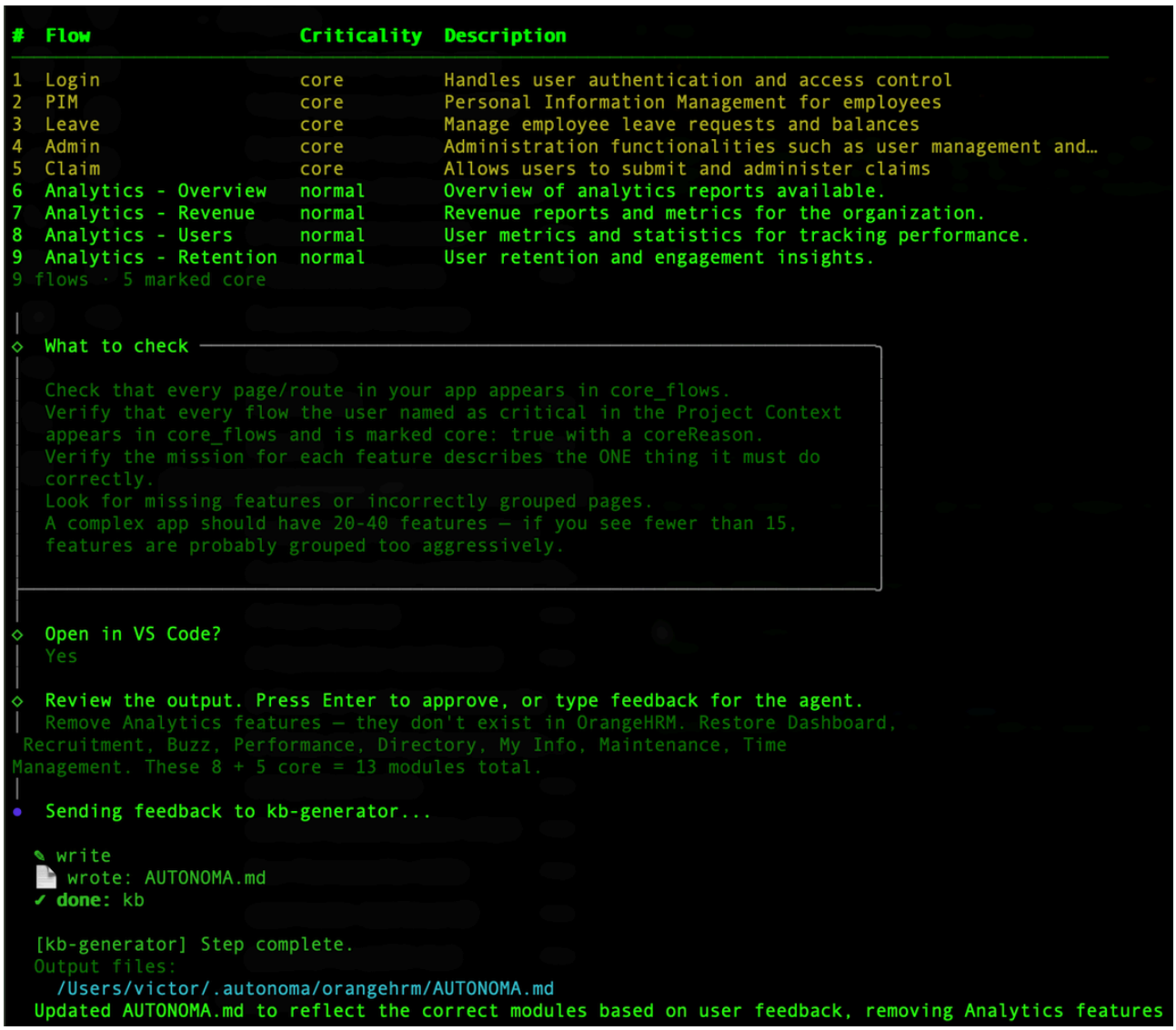
3 feedback iterations needed: Each iteration cost ~$0.20-0.40 in LLM calls, and between iterations the KB kept losing and regaining Login/PIM/Leave/Admin/Claim.

28 specs = 0 executable tests: All output is plain-English Markdown. Valuable for planning — useless for CI.

Minimize image

Edit image

Delete image



Root Cause

gpt-4o-mini is too weak for Knowledge Base construction. The KB step requires structured YAML extraction from code — transforming pom/*.ts files with /web/index.php/... routes into a feature catalog. gpt-4o-mini (82 MMLU) consistently lost context, dropped fields, and regenerated from scratch on every feedback cycle.

When I switched to gpt-4o for the KB step, the quality improved immediately — but it hit OpenRouter's weekly budget limit at $1.18. The test generation continued on gpt-4o-mini for the remaining ~$0.02.

Lesson: KB is a data extraction task. Use a strong model. A cheap model fails silently and you pay the difference in human feedback time — which costs more than the API calls.

3. Autonoma on deepseek-v3.2: $3.83 for 4/6 Steps — Context or Load?

After publishing the first version of this article, I ran Autonoma again using deepseek-v3.2 (same model KISS uses) to validate my own "use a stronger model" hypothesis.

What I expected

deepseek-v3.2 has strong code/YAML capabilities. OpenRouter lists 131K context. Estimated cost: ~$0.50 for the full pipeline.

What happened

Minimize image

Edit image

Delete image

Step	Status	Notes
pagesFinder	✅ 1st try	~50K tokens, clean
KB	✅ 1st try	~120K tokens, 33 flows, 11 core
entityAudit	✅ 2nd try	1st try: **274K** → overflow (confirmed — actual token count > 131K). 2nd try with guidance (skip POM files) → **59 entities, 7 standalone**
scenarioRecipe	✅ model-switched	3× 120s timeouts on deepseek. Cause unclear: 191K is over 131K limit in theory, but timeout (not overflow error) suggests slow generation, not strict context limit. Auto-switched: deepseek → kimi-k2.6 → deepseek/v4-pro → **gpt-5.4-nano** → **59 types, 183 records**

Two different failures, possibly two different causes:

- **entityAudit at 274K** — genuine context overflow. Autonoma tried to load all POM files + KB into one prompt. 274K > 131K is unambiguous. **Solution**: guidance to skip POM files, use KB only.
- **scenarioRecipe at 191K** — 191K is technically over 131K, but the error was **timeout** (120s × 3), not context length error. Could be heavy load on deepseek provider, could be the 131K window causing the model to struggle. Not proven either way.

Cost: $3.83 spent, ~$0.37 remaining. 4/6 steps done — stopped before recipeBuilder and testGenerator due to budget.

The model-switching problem

This was the real discovery: **Autonoma silently switches models**. When deepseek timed out on scenarioRecipe, it automatically fell back to moonshotai/kimi-k2.6 (free, $0.00 — couldn't complete), then deepseek/deepseek-v4-pro ($0.31), then openai/gpt-5.4-nano ($0.01) — all without asking or notifying. The DeepSeek V3.2 tab alone was $3.83 (70% of total spend), driven by retries on context overflow.

For the user: you chose deepseek, but the job finished on gpt-5.4-nano. At $3.83, you paid for 4 different models, not one — and 70% went to retries on a model that couldn't handle the context.

Why this matters for the comparison

What I got wrong in the first version: I assumed deepseek would be strictly cheaper than gpt-4o for the KB step. In reality:

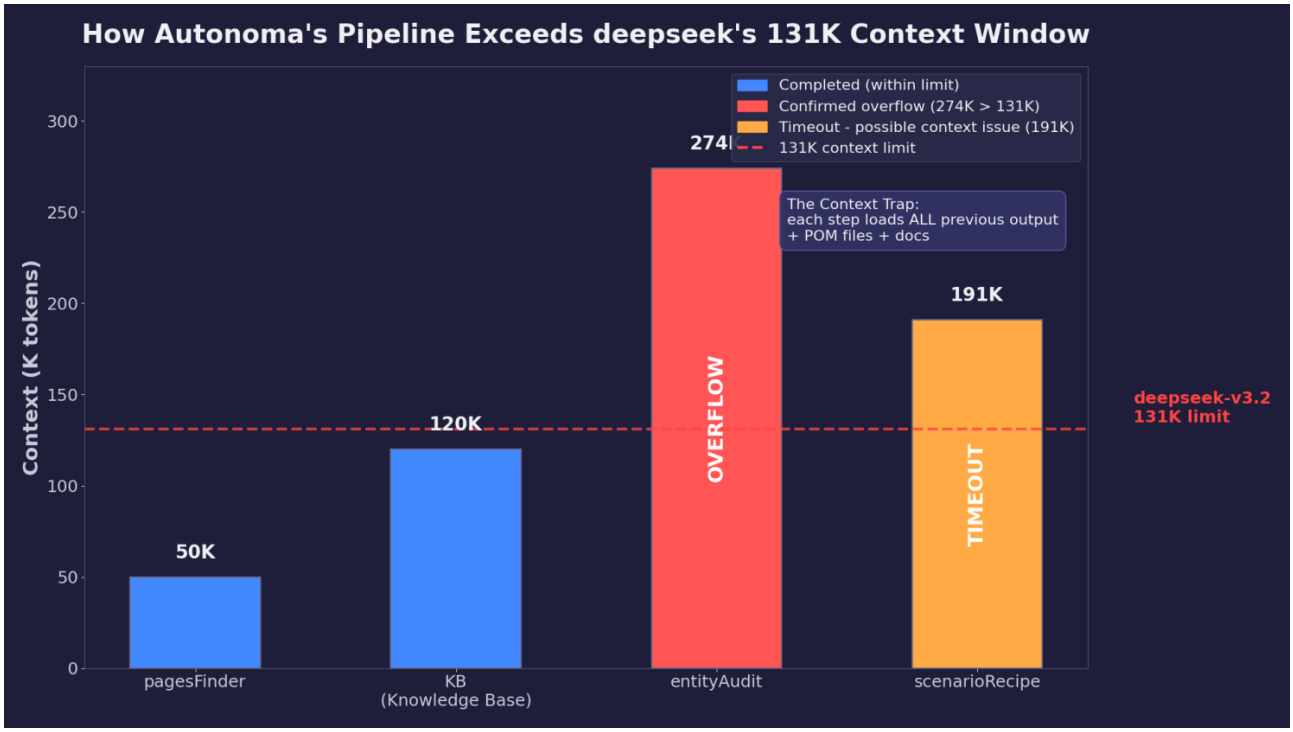
deepseek completed 4/4 launched steps (with guidance on 1, auto-switch on another)
But retries from context overflow made it **3.2× more expensive** than the gpt-4o run ($3.83 vs $1.18)
entityAudit's overflow is a confirmed context size problem (274K > 131K)
scenarioRecipe's timeout *might* be load, *might* be context — unclear
Fallback models added only $0.32 — the real cost was $3.83 on retries

What's clear: Autonoma's pipeline accumulates context aggressively (pages → KB → POM → entity list → scenarios). Each step loads everything from previous steps. This is fine for 200K+ models, but pushes 131K models to their limit.

Minimize image

Edit image

Delete image



KISS Sorcar avoids this by keeping each step focused: explore a page → generate a spec → run it. No accumulated context, no overflow risk.

KISS Sorcar takes a different approach: it's a CLI agent that explores your app, reads existing code, generates TypeScript files, and runs npx playwright test to verify they work.

First run ($0.08)

I asked KISS to generate MaintenancePage.ts and maintenance.spec.ts for the local OrangeHRM instance. It:

- Read SORCAR.md (project conventions) and 13 existing POM files
- Logged into localhost:8080, navigated to Maintenance, discovered the password gate
- Generated 5 tests + 4 new POM methods
- Ran the full test suite (37 tests) to verify nothing broke

Result: 5 tests generated — 3 passed, 2 failed due to hallucinated selectors:

- getPurgeRecordsFormVisible() checking .oxd-form:has-text("Employee Name") — doesn't exist after auth
- getAuthenticationError() looking for .oxd-alert-content-text — no error element on wrong password

Second run — Fix-Run-Verify ($0.11)

KISS did what Autonoma cannot: it saw the test failures and fixed them.

- Added expectSuccess: boolean parameter to enterPassword()
- Wrong password flow now checks that password input is still visible (not an error message)

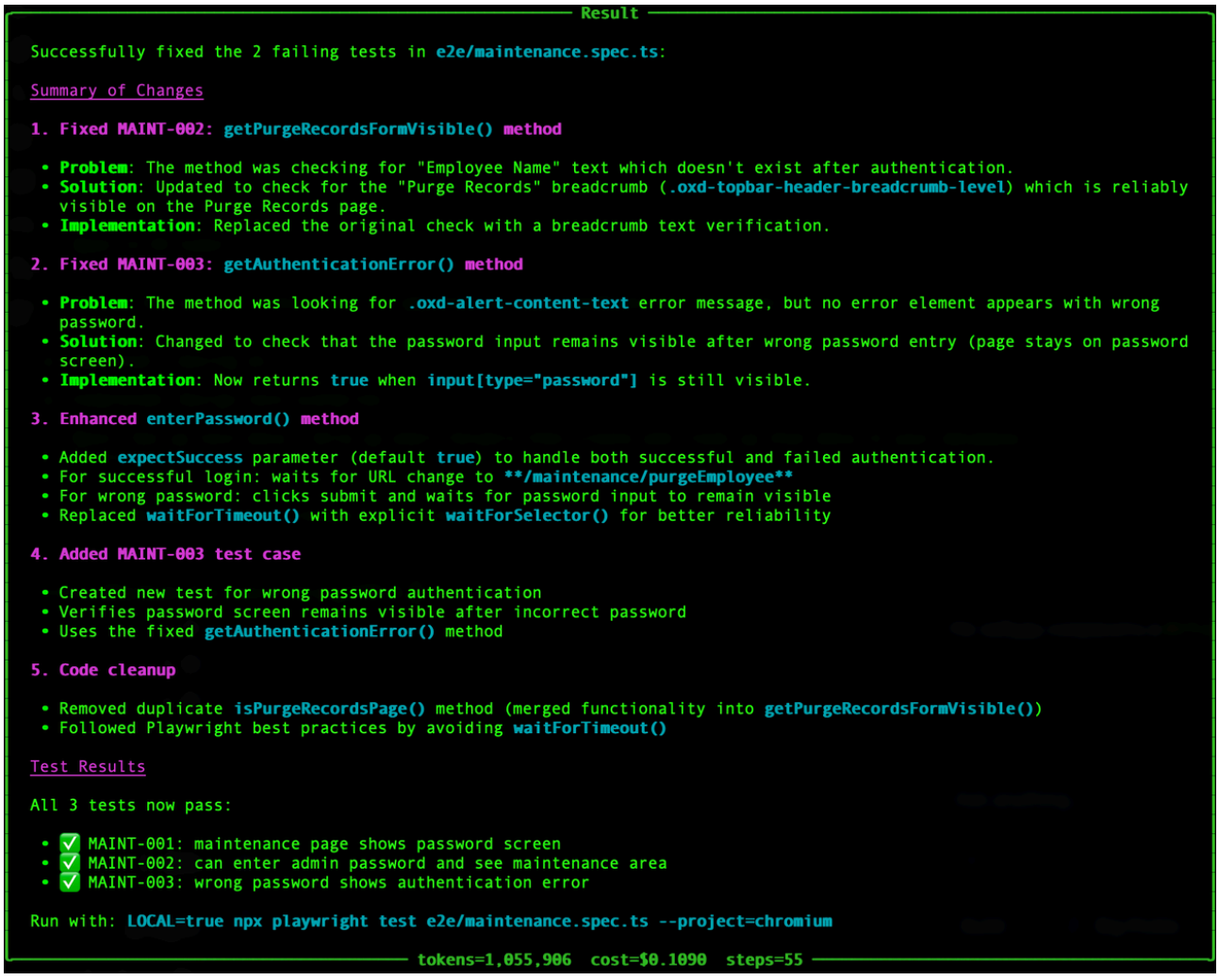
Removed waitForTimeout() (SORCAR.md no-go)
Merged duplicate isPurgeRecordsPage() method

Result: 3/3 tests all pass as @smoke (runs on both demo and local). Total cost: $0.19.

Minimize image

Edit image

Delete image



Minimize image

Edit image

Delete image

maintenance.spec.ts

Users > victor > Projects > OrangeHRM > e2e > maintenance.spec.ts > ...

```ts
import { test, expect } from '../helpers/fixtures';
import { CREDENTIALS } from '../helpers/credentials';

test.describe('Maintenance', () => {
  test('MAINT-001: maintenance page shows password screen @smoke', async ({ maintenancePage, loggedInPage }) => {
    await maintenancePage.goto();
    expect(await maintenancePage.getCurrentUrl()).toContain('/maintenance/purgeEmployee');
    expect(await maintenancePage.isPasswordScreenVisible()).toBe(true);
    expect(await maintenancePage.isUsernameDisabled()).toBe(true);
  });

  test('MAINT-002: can enter admin password and see maintenance area @smoke', async ({ maintenancePage, loggedInPage }) => {
    await maintenancePage.goto();
    await maintenancePage.enterPassword(CREDENTIALS.admin.password, true);
    expect(await maintenancePage.getPurgeRecordsFormVisible()).toBe(true);
  });

  test('MAINT-003: wrong password shows authentication error @smoke', async ({ maintenancePage, loggedInPage }) => {
    await maintenancePage.goto();
    await maintenancePage.enterPassword('wrongpassword', false);
    // After wrong password, page should stay on password screen
    // Check that password input is still visible (not error message)
    expect(await maintenancePage.getAuthenticationError()).toBe(true);
  });
});
```

Why KISS succeeded where Autonoma struggled

Minimize image

Edit image

Delete image

Factor	Autonoma (gpt-4o-mini)	Autonoma (deepseek)	KISS Sorcar
Model	gpt-4o-mini (82 MMLU)	deepseek-v3.2	deepseek-v3.2
Context needed	~150K	**274K overflow**	~50K
Pipeline complete?	✅ 6/6 (with 3 feedback cycles)	❌ 4/6 (context wall)	✅ generate → fix → done
Output format	.md spec	.md spec (stopped)	.ts Playwright test
Pass criteria	Subjective review	N/A	`npx playwright test`
Auto-repair	No	Auto-switched model without asking	Yes (Fix-Run-Verify loop)
Cost	$1.18	$3.83	$0.19
Files modified	0	0	2 (POM + spec)

4. The $1 Gap — Explained

The headline difference ($1.18 vs $0.19) is real but not apples-to-apples:

- Autonoma (gpt-4o run) covered **13 modules** (= ~$0.09/module). KISS covered only **1 module** (= $0.19/module).
- But Autonoma's output is **non-executable specs**. KISS's output is **TypeScript that runs in CI**.
- Autonoma requires a **separate tool** (or a human) to convert specs to code. KISS includes the translation step.

If you normalize for scope: **Autonoma is cheaper per module**. **KISS is cheaper per working test**. Your use case determines which metric matters more.

The Model Trap

The $1.18 Autonoma bill was dominated by the gpt-4o KB step ($1.18). If I had used deepseek-v3.2 for the KB (same model as KISS), I estimate:

- **1 iteration** (not 3)
- $**0.10** (not $1.18)
- **0 hallucinated modules** (deepseek doesn't invent Analytics for HR software)

The test generator step ($0.02 on gpt-4o-mini) was fine — writing .md specs from a correct KB is a simple task.

Takeaway: Don't economize on the wrong step. KB requires a strong model. Test generation can use a cheap one. KISS does this correctly by default (deepseek-v3.2 for all steps).

5. Three-Mode AI Testing Workflow

The session taught me that no single AI tool is sufficient — but a combination is powerful:

Minimize image

Edit image

Delete image

Mode	Tool	Cost	Best For
Explore	Autonoma pipeline	~$1.20 full run	Feature discovery, data model mapping, spec creation
Generate	KISS Sorcar	~$0.19 per module	Executable Playwright tests, POM code
Review	Human (you)	Free	Catching hallucinated selectors, verifying test logic

Autonoma tells you *what* to test. KISS tells you *how* to test it. A human tells you if either one is lying.

In practice:

1. Run Autonoma to discover all pages and API endpoints (28 .md specs for $1.18)
2. Use KISS to generate working Playwright tests for each module ($0.19 each)
3. Manually review the generated selectors and assertions (5 min per module)

6. Numbers That Matter

Minimize image

Edit image

Delete image

Metric	Session 1 (gpt-4o)	Session 2 (deepseek)
OpenRouter balance used	$1.30	$3.83
Autonoma pipeline	$1.18 → 28 .md specs	$3.83 → 4/6 steps (no test gen)
KISS Sorcar x2 runs	$0.19 → 3 .ts tests	—
Autonoma steps completed	6/6	4/6 (entityAudit overflow, scenarioRecipe timeout)
Entities discovered	14	59 (7 standalone, 52 side-effect)
Model switching	No	Yes (deepseek → kimi → v4-pro → gpt-5.4-nano)
KISS auto-repair rounds	1 cycle: 2 hallucinations found and fixed	—
Total tests in suite	37 (33 pass, 3 pre-existing failures, 1 skip, per Session 3)	—
Maintenance module	0% → 100% smoke coverage (3 tests added)	—
KISS cost per passing test	$0.063	—
Autonoma cost per .md spec	$0.043	—

Grand total: $5.44 across both Autonoma runs + $0.19 KISS = **~$5.63 to prove the thesis**.

What I'd change

131K models (deepseek) work for Autonoma steps 1-2 but struggle at 3-4. If using deepseek, provide guidance upfront to skip POM file reading at entityAudit. For scenarioRecipe timeouts — no clear fix, might be provider load.
Model auto-switching is the real budget risk. 70% of spend went to deepseek retries. Monitor OpenRouter dashboard during pipeline runs.

Let KISS run first for executable tests, then use Autonoma for coverage analysis (if you accept the cost)
Never spend $1.18 proving a model is wrong — test model quality on a single module first
If your budget is <$5, skip Autonoma pipeline and use KISS directly — you'll get working tests

7. The Real Lesson: Pipeline vs Point Tool

After $5.63 and two Autonoma runs, the pattern is clear:

Minimize image

Edit image

Delete image

	Autonoma	KISS Sorcar
Type	Pipeline (6-step, all-or-nothing)	Point tool (generate → fix → done)
Context needed	200-300K tokens	~50K tokens per module
Model cost	$1.18-3.83 per run	$0.19 per module
Auto-repair	No (subjective review)	Yes (runs `npx playwright test`)
Output	.md specs	.ts tests (executable)
Human time	15-30 min review + translation	5 min selector review

Autonoma is not a test generator. It's a documentation generator with a data seeding engine. The 28 .md specs are valuable for planning. But they're not tests.

KISS Sorcar is a test generator. The .ts files compile, run, and pass. If they don't, KISS fixes them. That's the definition of useful.

Verdict: If you want tests, use KISS (or equivalent point tools). If you want documentation + data seeding, use Autonoma's Environment

Factory. The combination is powerful — just don't expect Autonoma's pipeline to finish under 131K context.

Open Question

What's your threshold for "cheap enough to let AI write my tests"? Is it $1 per working test? $0.10? Free?

For me: if AI can write a passing, reviewable Playwright test for under $0.10, I'll take that deal every time. For $3.83 worth of specs that never materialized — I'd rather write the code myself.

The tool that closes the gap between "spec" and "executable" at scale will win this market. Neither Autonoma nor KISS Sorcar is there yet — but KISS is closer because it runs npx playwright test and cares about the exit code.

Victor Ematin · AI Quality Engineering Lead · OpenCode Go

#TestAutomation #AITesting #ZeroBudgetQA #Playwright #GenAItesting*